

Νευρωνικά Δίκτυα, 3η Εργασία

Ιωάννης Κέλπης AEM:10300

Ιανουάριος 2024

Περίληψη

Σκοπός της εργασίας είναι να γραφεί και να μελετηθεί ένα Radial Basis Function (RBF) νευρωνικό δίκτυο. Η γλώσσα που επιλέχθηκε για το συγκεκριμένο project είναι η Python (version 3.11.5).

Η βάση δεδομένων που επιλέχθηκε είναι η Cifar10, χρησιμοποιήθηκε ολόκληρη.

Η υλοποίηση του Νευρωνικού Δικτύου, έγινε με την βοήθεια της βιβλιοθήκης scikit-learn.

Αφού δημιουργήθηκε το νευρωνικό δίκτυο, δοκιμάστηκαν και μελετήθηκαν διάφοροι τρόποι εκπαίδευσης, και διάφορες τιμές τόσο για τον αριθμό των νευρώνων στο κρυφό επίπεδο όσο και για τον αριθμό επαναλήψεων του αλγορίθμου βελτιστοποίησης και των κέντρων του δικτύου, παράμετροι που επηρεάζουν σημαντικά την ακρίβεια του νευρωνικού δικτύου.

Chapter 1

Παρουσίαση και ανάλυση του κώδικα

Σε αυτό το κεφάλαιο θα μελετήσουμε και θα αναλύσουμε τον κώδικα που είναι υπεύθυνος για την δημιουργία και την εκπαίδευση του Νευρωνικού μας δικτύου.

Πρώτο και σημαντικότερο βήμα, είναι η εισαγωγή των απαραίτητων βιβλιοθηκών.

- "numpy": Χρησιμοποιείται για αριθμητικούς υπολογισμούς σε Python. Στον κώδικα, χρησιμοποιείται κυρίως για την επεξεργασία δεδομένων και τον υπολογισμό πολλών πολύπλοκων μαθηματικών εκφράσεων.
- scikit-learn: είναι μια ανοιχτού κώδικα βιβλιοθήκη που παρέχει εργαλεία για μηχανική μάθηση και ανάλυση δεδομένων στην πλατφόρμα της Python. Παρέχει κάποιες από τα βασικές υλοποιήσεις που θα χρησιμοποιήσουμε στην εργασία μας, όπως το μοντέλο του MLP κατηγοριοποιητή.
neural_network: Παρέχει την υλοποίηση του MLP (Multi-Layer Perceptron) Classifier που χρησιμοποιείται για την εκπαίδευση του νευρωνικού δικτύου.
metrics: Χρησιμοποιείται για τον υπολογισμό της ακρίβειας (accuracy) του μοντέλου.
cluster: Παρέχει την υλοποίηση του K-Means clustering, που χρησιμοποιείται για την ομαδοποίηση των δεδομένων.
metrics.pairwise: Παρέχει συναρτήσεις για τον υπολογισμό πυρήνων (όπως ο RBF kernel) μεταξύ δεδομένων.
- TensorFlow: Χρησιμοποιείται για τη φόρτωση του CIFAR-10 dataset. Η TensorFlow είναι μια βιβλιοθήκη για μηχανική μάθηση και βαθιά μάθηση.
- time: Χρησιμοποιείται για τη μέτρηση του χρόνου εκτέλεσης της εκπαίδευσης του μοντέλου.
- "matplotlib.pyplot": παρέχει εργαλεία για την γραφική αναπαράσταση των αποτελεσμάτων του δικτύου μας.

```
import numpy as np
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import accuracy_score
from sklearn.cluster import KMeans
from sklearn.metrics.pairwise import rbf_kernel
import tensorflow as tf
import matplotlib.pyplot as plt
import time
```

Ξεκινάμε φορτώνοντας το dataset Cifar10 με την βοήθεια της Tensorflow. Ταυτόχρονα, επεξεργαζόμαστε τα δεδομένα μας, αλλάζοντας το σχήμα των εικόνων και κανονικοποιώντας τις τιμές των pixel τους στο διάστημα [0,1]. Έπειτα, κανονικοποιούμε τα χαρακτηριστικά, προκειμένου να έχουν κοινή κλίμακα. Αυτό το βήμα είναι ιδιαίτερα σημαντικό για τα μοντέλα που βασίζονται σε αλγόριθμους που επηρεάζονται από την κλίμακα των χαρακτηριστικών, όπως τα SVM. Η κανονικοποίηση αυτή γίνεται με τη μετατροπή των τιμών των χαρακτηριστικών έτσι ώστε να έχουν μέση τιμή 0 και τυπική απόκλιση 1, με την βοήθεια του Standard Scaler.

```
# Load CIFAR-10 dataset using TensorFlow
(X_train, y_train), (X_test, y_test) =
tf.keras.datasets.cifar10.load_data()

# Preprocess the data
X_train = X_train.reshape(X_train.shape[0], -1)
X_test = X_test.reshape(X_test.shape[0], -1)
X_train = X_train.astype('float32') / 255.
X_test = X_test.astype('float32') / 255.

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

Στην συνέχεια χρησιμοποιούμε τον αλγόριθμο KMeans για να ομαδοποιήσουμε τα δεδομένα εκπαίδευσης (X_train) σε 100 clusters με βάση την ομοιότητα των δεδομένων, και αποθηκεύουμε τα κέντρα των clusters.

```
# Cluster the data using KMeans
kmeans = KMeans(n_clusters=100, random_state=42).fit(X_train)
centers = kmeans.cluster_centers_
```

Έπειτα υπολογίζουμε τον πυρήνα RBF για τα δεδομένα εκπαίδευσης και ελέγχου, με την βοήθεια των κέντρων που αποθηκεύσαμε στο προηγούμενο βήμα και ορίζουμε την παράμετρο gamma (γ).

Η παράμετρος gamma είναι μια υπερ-παράμετρος που είναι υπεύθυνη για το πλάτος του πυρήνα. Μια μικρότερη τιμή του gamma οδηγεί σε έναν πυρήνα μεγαλύτερου πλάτους (που μπορεί να οδηγήσει σε υπερεκπαίδευση) ενώ μια μεγαλύτερη τιμή του gamma οδηγεί σε έναν πυρήνα μικρότερου πλάτους (που μπορεί να οδηγήσει σε υποεκπαίδευση).

Σε αυτήν την περίπτωση, το gamma έχει οριστεί σε $1.0 / (2 * 100 ** 2)$ επειδή υπάρχουν 100 ομάδες (clusters) στον αλγόριθμο ομαδοποίησης KMeans που χρησιμοποιείται στον κώδικα. Η τιμή του gamma επιλέγεται έτσι ώστε ο πυρήνας RBF να έχει ένα πλάτος που είναι ανάλογο με τη μέση απόσταση μεταξύ των σημείων δεδομένων και των κέντρων των ομάδων. Συγκεκριμένα, το gamma ορίζεται σε $1.0 / (2 * d ** 2)$, όπου d είναι η μέση απόσταση μεταξύ των σημείων δεδομένων και των κέντρων των ομάδων. Επειδή υπάρχουν 100 ομάδες σε αυτήν την περίπτωση, το d είναι περίπου ίσο με $1/100$, και συνεπώς το gamma ορίζεται σε $1.0 / (2 * 100 ** 2)$.

```
# Compute the RBF kernel matrix
gamma = 1.0 / (2 * 100 ** 2)
K_train = rbf_kernel(X_train, centers, gamma=gamma)
K_test = rbf_kernel(X_test, centers, gamma=gamma)
```

Ο παρακάτω κώδικας, δημιουργεί έναν MLP κατηγοριοποιητή με πυρήνα RBF ο οποίος εκπαιδεύεται με τα δεδομένα εκπαίδευσης `K_train` και `y_train` και κατηγοριοποιεί το test data `K_test`. Η συνάρτηση ενεργοποίησης που χρησιμοποιείται είναι η σιγμοειδής,

$$f(x) = 1/(1 + \exp(-x))$$

Η παράμετρος `solver` έχει την τιμή `'lbfgs'` αυτό σημαίνει ότι χρησιμοποιούμε τον αλγόριθμο Limited-memory Broyden-Fletcher-Goldfarb-Shanno (L-BFGS) προκειμένου να βελτιστοποιήσουμε τα βάρη του δικτύου. Το δίκτυο μας έχει ένα κρυφό επίπεδο με 100 νευρώνες. Η παράμετρος `random_state` έχει οριστεί σε 42, πράγμα που σημαίνει ότι ο random number generator που χρησιμοποιείται στον αλγόριθμο βελτιστοποίησης έχει αρχικοποιηθεί με μια σταθερή τιμή αρχικής τιμής 42.

Σημειώνεται πως σε αυτό το κομμάτι του κώδικα ελέγχουμε και τον χρόνο που παίρνει στο νευρωνικό μας δίκτυο να εκπαιδευτεί

```
# Train the RBF Neural Network
start_time = time.time()

rbf_nn = MLPClassifier(hidden_layer_sizes=(100,), activation='logistic',
                        solver='lbfgs', max_iter=500, random_state=42)
rbf_nn.fit(K_train, y_train)

end_time = time.time()
```

Αφού εκπαιδέυσουμε το νευρωνικό μας δίκτυο, τώρα το αξιολογούμε! Σε αυτό το σημείο, ελέγχουμε την ακρίβεια που έχει τόσο στο στάδιο της εκπαίδευσης όσο και στο στάδιο ελέγχου, εκτυπώνοντας τα αποτελέσματα.

```
# Evaluate the RBF Neural Network
y_train_pred = rbf_nn.predict(K_train)
train_accuracy = accuracy_score(y_train, y_train_pred)
print(f'The accuracy of the RBF Neural Network on the training set is {train_accuracy:.2%}.')

y_test_pred = rbf_nn.predict(K_test)
test_accuracy = accuracy_score(y_test, y_test_pred)
print(f'The accuracy of the RBF Neural Network on the test set is {test_accuracy:.2%}.')
```

Τέλος οι παρακάτω γραμμές κώδικα μας επιτρέπουν να εμφανίσουμε επιλεγμένες εικόνες αφού εκπαιδέυσουμε το νευρωνικό μας δίκτυο, προκειμένου να εξετάσουμε κάποια παραδείγματα ορθής και εσφαλμένης κατηγοριοποίησης.

```

# Get the predicted labels for the test set
y_pred = rbf_nn.predict(K_test)

# Get the indices of the misclassified images
misclassified_indices = [i for i, pred in enumerate(y_pred)
                        if pred != y_test[i]]

# Plot the first 10 misclassified images
fig, axs = plt.subplots(2, 5, figsize=(15, 3))
axs = axs.ravel()
for i in range(5):
    axs[i].imshow(X_test[misclassified_indices[i]].reshape(32, 32, 3))
    axs[i].set_title(f"True-label: {y_test[misclassified_indices[i]][0]},
    ----, Predicted-label: {y_pred[misclassified_indices[i]]}")

# Get the indices of the correctly classified images
correct_indices = [i for i, pred in enumerate(y_pred)
                  if pred == y_test[i]]

# Plot the first 10 correctly classified images
for i in range(5):
    axs[i+5].imshow(X_test[correct_indices[i]].reshape(32, 32, 3))
    axs[i+5].set_title(f"True-label: {y_test[correct_indices[i]][0]},
    ----, Predicted-label: {y_pred[correct_indices[i]]}")
plt.show()

```

Chapter 2

Έκθεση των αποτελεσμάτων

Σε αυτό το κεφάλαιο, θα παρουσιαστούν κάποια αποτελέσματα που προέκυψαν από την δημιουργία του Νευρωνικού μας δικτύου. Αναλυτικότερα θα γίνει αναφορά:

- σε μερικά παραδείγματα ορθής και εσφαλμένης κατηγοριοποίησης
- στα ποσοστά επιτυχίας του νευρωνικού μας δικτύου, στα στάδια της εκπαίδευσης (training) και του ελέγχου (testing)
- στον χρόνο που χρειάζεται το νευρωνικό μας δίκτυο για να εκπαιδευτεί, για διαφορετικές παραμέτρους του δικτύου
- στα διαφορετικά ποσοστά ακριβείας που επιτυγχάνουμε με μεταβολή κάποιων παραμέτρων εκπαίδευσης, όπως του αριθμού των κρυφών νευρώνων, και των κέντρων.

Τέλος, θα γίνει εν συντομία και μια σύγκριση μεταξύ της απόδοσης του RBF νευρωνικού δικτύου, σε σχέση με τους κατηγοριοποιητές πλησιέστερου γείτονα (KNN) και πλησιέστερου κέντρου (Nearest Centroid) της ενδιάμεσης εργασίας.

2.1 Ορθή - Εσφαλμένη κατηγοριοποίηση

Οι παρακάτω κατηγοριοποιήσεις, αντιστοιχούν σε υλοποίηση του δικτύου μας με 100 κρυφούς νευρώνες, και την παράμετρο `max_iter` να είναι ίση με 500. Παρακάτω, παρουσιάζονται με την σειρά:

- 5 παραδείγματα ορθής εσφαλμένης κατηγοριοποίησης του νευρωνικού μας δικτύου.
- το ποσοστό επιτυχίας του δικτύου μας για το training set και το test set.

Όπως είναι φυσικό, το νευρωνικό μας δίκτυο δεν μπορεί να κατηγοριοποιήσει όλες τις εικόνες με ορθό τρόπο. Βλέπουμε πως μετά την διαδικασία της εκπαίδευσης, το μοντέλο μας φτάνει να έχει ακρίβεια της τάξης του 22.95%. Το ποσοστό accuracy είναι πολύ χαμηλό, επομένως είναι αναμενόμενο να βρεθούν εικόνες οι οποίες αντιστοιχήθηκαν σε λάθος κλάση, και παρουσιάζονται μερικές από αυτές!

2.2 Εκπαίδευση του Νευρωνικού μας δικτύου για διαφορετικό αριθμό του `max_iter`

Στο προηγούμενο κομμάτι της εργασίας μας, είχαμε ορίσει την παράμετρο `max_iter` ίση με 500, και πραγματοποιήσαμε τις μετρήσεις μας. Στο output όμως του κώδικα, λάβαμε το παρακάτω warning: "STOP: TOTAL NO. of ITERATIONS REACHED LIMIT."

Για να κατανοήσουμε καλύτερα τι σημαίνει αυτό το μήνυμα, δίνουμε παρακάτω την σημασία αυτής της παραμέτρου. Η παράμετρος `max_iter` αναφέρεται στο μέγιστο αριθμό επαναλήψεων που απαιτείται για να συγκλίνει ο αλγόριθμος βελτιστοποίησης. Σχετίζεται με τη διαδικασία της βελτιστοποίησης κατά τη διάρκεια της εκπαίδευσης, όπου ο αλγόριθμος ενημερώνει επαναληπτικά τις παραμέτρους του μοντέλου (βάρη) για να ελαχιστοποιήσει τη συνάρτηση απώλειας. Η διαδικασία εκπαίδευσης περιλαμβάνει



Figure 2.1: Classification Examples

πολλαπλές επαναλήψεις, και το *max_iter* ορίζει ένα ανώτατο όριο για το πόσες επαναλήψεις μπορεί να πραγματοποιήσει ο αλγόριθμος.

Όπως καταλαβαίνουμε λοιπόν, το πρόγραμμα μας προειδοποιεί ότι ο αλγόριθμος βελτιστοποίησης, δεν "προλαβαίνει" να συγκλίνει, και μας προτρέπει να αυξήσουμε τον επιτρεπτό αριθμό επαναλήψεων. Πράγματι, αυξάνουμε σταδιακά την παράμετρο *max_iter*, και όπως βλέπουμε στο παρακάτω πινακάκι, το ποσοστό της ακρίβειας, τόσο στο training set όσο και στο test set ανεβαίνει μέχρι το σημείο που η παράμετρος μας παίρνει την τιμή 5000, και καταλαβαίνουμε ότι αυτή είναι μια ικανοποιητική τιμή.

<i>max_iter</i> value	accuracy (train set - test set)
1000	26.91% - 27.08%
2000	30.84% - 31.05%
3000	41.4% - 41.83%
4000	41.77% - 41.98%
5000	42.47% - 43.05%

Table 2.1: Αριθμός επαναλήψεων και ακρίβεια

Αξίζει επίσης να σημειωθεί, πως με την αύξηση του αριθμού επαναλήψεων, μεγάλωνε όπως είναι φυσικό και ο χρόνος που χρειαζόταν το νευρωνικό μας δίκτυο προκειμένου να εκπαιδευτεί!

2.3 Εκπαίδευση του Νευρωνικού με διαφορετικό αριθμό νευρώνων στο κρυφό επίπεδο

Αφού στο προηγούμενο βήμα, διαπιστώσαμε ποια πρέπει να είναι η τιμή της παραμέτρου *max_iter*, τώρα θα εξετάσουμε πως επηρεάζεται το νευρωνικό μας δίκτυο, όταν μεταβάλλουμε τον αριθμό των νευρώνων στο κρυφό επίπεδο. Στο παρακάτω πινακάκι, παραθέτονται ο χρόνος εκπαίδευσης του δικτύου, για διαφορετικές τιμές της μεταβλητής *hidden_size* που δηλώνει τον αριθμό των νευρώνων στο κρυφό επίπεδο. Πιο συγκεκριμένα, πραγματοποιήσαμε ένα Grid Search, για διάφορες τιμές νευρώνων στο κρυφό επίπεδο, προκειμένου να βρούμε την πιο αποδοτική τιμή που μπορεί να πάρει η παράμετρος *hidden_layer_sizes*. Όπως φαίνεται και παρακάτω, η βέλτιστη τιμή που μπορεί να πάρει η υπερπαράμετρος αυτή είναι οι 200 κρυφοί νευρώνες.

Το Grid Search είναι μια τεχνική που χρησιμοποιείται για την αναζήτηση του βέλτιστου συνδυασμού

υπερπαραμέτρων για ένα μοντέλο. Ορίζουμε ένα "grid" από πιθανούς συνδυασμούς υπερπαραμέτρων και εκτελούμε την εκπαίδευση και αξιολόγηση του μοντέλου για κάθε συνδυασμό. Στη συνέχεια, επιλέγουμε τον συνδυασμό που παρέχει τα καλύτερα αποτελέσματα. Οι παρακάτω μετρήσεις έγιναν για $\text{max_iter} = 5000$.

Neurons	Training Time
50	5.4min
100	13.2min
150	16mins
200	18.66min
300	29.56min

Table 2.2

Στον παραπάνω πίνακα παρατηρούμε πως ο χρόνος εκπαίδευσης του νευρωνικού μας δικτύου μεγαλώνει με την αύξηση των νευρώνων στο κρυφό επίπεδο. Αυτό μπορεί να οφείλεται σε διαφορετικούς λόγους. Αρχικά, με περισσότερους νευρώνες, αυξάνεται η υπολογιστική πολυπλοκότητα του δικτύου, καθώς υπάρχουν περισσότερες παράμετροι που πρέπει να ενημερώνονται κατά τη διάρκεια της εκπαίδευσης και απαιτούνται περισσότερες λειτουργίες για τον υπολογισμό των κλίσεων και των βαρών. Ακόμα μεγαλύτερα κρυφά στρώματα μπορεί να οδηγήσουν σε έναν πιο πολύπλοκο χώρο απώλειας, που πιθανόν να οδηγήσει σε πιο αργή σύγκλιση κατά τη βελτιστοποίηση. Ο αλγόριθμος βελτιστοποίησης (optimizer) ενδέχεται να απαιτεί περισσότερα βήματα για να βρει το ελάχιστο της συνάρτησης απώλειας (loss function).

Αξίζει επίσης να αναφερθεί, ότι στο *Grid Search* χρησιμοποιείται και η τεχνική *cross validation* (με την παράμετρο $cv=3$). Αυτό σημαίνει ότι κατά τη διαδικασία του *Grid Search*, ο αλγόριθμος εκπαιδεύεται και αξιολογεί το μοντέλο μας τρεις φορές, χρησιμοποιώντας διαφορετικά υποσύνολα του συνολικού συνόλου δεδομένων. Πιο συγκεκριμένα, το σύνολο δεδομένων διασπάται σε τρία υποσύνολα, και κάθε φορά το μοντέλο εκπαιδεύεται σε δύο από τα υποσύνολα και αξιολογείται στο υποσύνολο που δεν χρησιμοποιήθηκε κατά την εκπαίδευση. Η χρήση του $cv=3$ βοηθά στη διασφάλιση ότι οι εκτιμήσεις της απόδοσης του μοντέλου είναι πιο σταθερές και γενικεύονται καλύτερα σε διάφορα υποσύνολα του συνόλου δεδομένων μας. Οι τιμές που φαίνονται στο πινακάκι είναι ο μέσος όρος των 3 "folds" για κάθε αριθμό κρυφών νευρώνων.

Όπως βλέπουμε στην φωτογραφία παρακάτω, με την βοήθεια του grid search καταλήξαμε πως το νευρωνικό μας δίκτυο αποκτά την μέγιστη ακρίβεια για 300 κρυφούς νευρώνες. Η ακρίβεια αυτή στο test set είναι 43.28%!

```
The best number of hidden neurons is (300,).
The accuracy of the RBF Neural Network with (300,) hidden neurons on the test set is 43.28%.
```

Figure 2.2: Hidden Neurons Best

2.4 Επίδραση του αριθμού των κέντρων στο νευρωνικό δίκτυο

Θα εξηγήσουμε πρώτα με ποιον τρόπο λειτουργεί ο αλγόριθμος Kmeans. Ο KMeans είναι ένα αλγόριθμος ομαδοποίησης που χρησιμοποιείται στη μηχανική μάθηση για τον διαμοιρασμό ενός συνόλου δεδομένων σε K διακριτά, μη-επικαλυπτόμενα υποσύνολα (clusters). Ο στόχος του αλγορίθμου KMeans είναι να βρει τα κέντρα των clusters έτσι ώστε το άθροισμα των τετραγώνων των αποστάσεων μεταξύ των σημείων δεδομένων και των κέντρων των αντίστοιχων τους clusters να ελαχιστοποιείται. Αυτό έχει ως αποτέλεσμα να δημιουργηθεί ένα set από κέντρα των clusters, τα οποία αντιπροσωπεύουν τον "μέσο" των σημείων που ανήκουν στο εκάστοτε υποσύνολο. Αυτόν τον αλγόριθμο χρησιμοποιούμε λοιπόν για τον διαχωρισμό των δεδομένων μας σε υποσύνολα και την εξαγωγή των κέντρων τους.

Με την χρήση κάποιων σταθερών τιμών των υπερπαραμέτρων για το δίκτυο μας που μελετήσαμε παραπάνω, θα πειραματιστούμε τώρα με τον αριθμό των κέντρων του δικτύου μας. (Σημειώνεται πως μέχρι στιγμής είχαμε 100 clusters στις παραπάνω μετρήσεις). Επειδή όπως είδαμε η εκπαίδευση του δικτύου μας διαρκεί αρκετή ώρα με τις βέλτιστες παραμέτρους, για την μελέτη των κέντρων επιλέγουμε κάποιες άλλες τιμές για τις παραμέτρους `max_iter` και τον αριθμό των κρυφών νευρώνων, προκειμένου να μπορέσουμε να κάνουμε περισσότερα πειράματα (`max_iter` = 3000, 100 κρυφοί νευρώνες) . Με την μεταβολή της παραμέτρου `clusters` (ομάδες) στον κώδικα μας, θα αλλάζουμε κατάλληλα και την παράμετρο `gamma`.

Εξετάζοντας πρώτα την τιμή `n_cluster` = 50, διαπιστώνουμε πως το νευρωνικό δίκτυο, παρόλο που δεν έχει τις βέλτιστες τιμές των παραμέτρων, έχει ακρίβεια 44.99% στο `test set`, μεγαλύτερη δηλαδή από κάθε άλλη δοκιμή! Προκειμένου να κατανοήσουμε καλύτερα την συμπεριφορά του δικτύου, δοκιμάζουμε στο επόμενο πείραμα μας να αυξήσουμε την αριθμό των clusters στα 150. Το νευρωνικό δίκτυο αυτή την φορά έχει μειωμένη ακρίβεια, καταλαβαίνουμε λοιπόν ότι το δίκτυο συμπεριφέρεται καλύτερα για αριθμό clusters μικρότερο από το 100. Συνεχίζουμε τις δοκιμές για διάφορες τιμές και παρουσιάζουμε συγκεντρωτικά τα αποτελέσματα στο παρακάτω πινακάκι.

Clusters	Time	Accuracy train set - test set
150	703.26s	37.98% - 38.23%
60	442.02s	45.10% - 44.99%
50	502.13s	45.48% - 44.99%
40	460.58s	45.93% - 44.90%
30	469.56s	43.69% - 42.24%

Table 2.3: Different number of clusters

Όπως φαίνεται, το δίκτυο μας αποκτάει την μεγαλύτερη ακρίβεια, όταν η μεταβλητή `n_clusters` παίρνει τιμές γύρω από το 50! Όπως επίσης είναι λογικό, παρατηρείται μια μικρή αύξηση στον χρόνο εκπαίδευσης του δικτύου όσο μεγαλώνει ο αριθμός των διαθέσιμων κέντρων.

Τυχαία κέντρα Αντί να χρησιμοποιήσουμε τον αλγόριθμο ομαδοποίησης KMeans, μπορούμε να πειραματιστούμε χρησιμοποιώντας τυχαία σημεία από τα δεδομένα μας ως κέντρα. Αυτό μπορεί να είναι χρήσιμο για να δούμε πόσο καλά αποδίδει το δίκτυο με τυχαία επιλεγμένα κέντρα. Αυτό το πετυχαίνουμε τροποποιώντας λίγο τον κώδικα με τον παρακάτω τρόπο:

```
# Cluster the data randomly
random_indices = np.random.choice(len(X_train), size=100, replace=False)
centers = X_train[random_indices]

#Compute the RBF kernel matrix
gamma = 1.0 / (2 * 100 ** 2)
K_train = rbf_kernel(X_train, centers, gamma=gamma)
K_test = rbf_kernel(X_test, centers, gamma=gamma)
```

Figure 2.3: Random Centers

Στο συγκεκριμένο παράδειγμα έχουμε και πάλι 100 διαφορετικά κέντρα. Σε αυτή την περίπτωση, το νευρωνικό δίκτυο μας έχει ακρίβεια στο `train test` 43.75% και στο `test set` 43.18%. Στην συνέχεια, κάνουμε το ίδιο πράγμα, αυτή την φορά με 50 τυχαία κέντρα. Προς έκπληξη μας, διαπιστώνουμε πως αυτή την φορά, το νευρωνικό δίκτυο έχει την μέγιστη απόδοση που πετύχαμε στην εργασία, δηλαδή 49.9% `train test` και 48.06% στο `test set`!!!

```
The accuracy of the RBF Neural Network on the training set is 49.90%.
The accuracy of the RBF Neural Network on the test set is 48.06%.
The RBF Neural Network took 435.19 seconds to train.
```

Figure 2.4: 50 Random centers

2.5 Σύγκριση του νευρωνικού μας δικτύου με τους κατηγοριοποιητές KNN και KNC

2.5.1 K-Nearest Neighbor Classifier

Ο κατηγοριοποιητής K πλησιέστερου γείτονα, είναι ένας διάσημος αλγόριθμος μηχανικής μάθησης, ο οποίος μπορεί να διαχειριστεί προβλήματα regression και classification. Κατά την διάρκεια της φάσης εκπαίδευσης, ο KNN αποθηκεύει το dataset σαν αναφορά. Όταν έρχεται η ώρα να κάνει προβλέψεις, με βάση μια συγκεκριμένη ιδιότητα που θεωρεί ως απόσταση (συνήθως την ευκλείδεια απόσταση), υπολογίζει την απόσταση ανάμεσα στα δεδομένα που του δίνουμε ως input, και όλα τα παραδείγματα εκπαίδευσης. Έπειτα, ο αλγόριθμος ταυτοποιεί τους K (στην περίπτωση μας 1, 3) πλησιέστερους γείτονες στο input. Αν το πρόβλημα μας είναι κατηγοριοποίησης ο αλγόριθμος αναθέτει την πιο συχνή κλάση που συναντάει στους γείτονες ως την πρόβλεψη του για το input data.

2.5.2 Nearest Centroid Classifier

Ο κατηγοριοποιητής πλησιέστερου κέντρου, είναι και αυτός ένας αλγόριθμος μηχανικής μάθησης, τον οποίο μπορούμε να εφαρμόσουμε σε προβλήματα κατηγοριοποίησης. Η κάθε κλάση του dataset, αναπαριστάται από το κέντρο της (δηλαδή τον "μέσο όρο" μιας τιμής που επιλέξαμε). Έτσι, όταν περάσουμε στον κατηγοριοποιητή κάποιο input, αυτός θα το αναθέσει σύμφωνα με την κλάση στην οποία ανήκει το πλησιέστερο κέντρο από το input data.

Αφού υλοποίησα λοιπόν τους παραπάνω κατηγοριοποιητές στο περιβάλλον της python, συνέκρινα την ακρίβεια τους, και όπως είναι φανερό από τον πίνακα παρακάτω, ο κατηγοριοποιητής πλησιέστερου γείτονα για $k = 1$, είναι ο πιο αποτελεσματικός και ακριβής, ενώ ακολουθούν με την σειρά ο κατηγοριοποιητής πλησιέστερου γείτονα για $k = 3$ και ο κατηγοριοποιητής πλησιέστερου κέντρου

Αλγόριθμος	Accuracy
KNN ($k = 1$)	35.39%
KNN ($k=3$)	33.03%
KNC	27.74%

Όπως είναι φανερό, η ακρίβεια του νευρωνικού μας δικτύου ξεπερνάει αυτήν των κατηγοριοποιητών της ενδιάμεσης εργασίας. Πιο συγκεκριμένα, παρατηρείται μια διαφορά στην ακρίβεια της τάξης 10-15%! Αυτό μπορεί να συμβαίνει για διάφορους λόγους, ενδεικτικά παρατίθενται μερικοί παρακάτω:

- Τα νευρωνικά δίκτυα εισάγουν από μόνα τους μη γραμμικότητα μέσω συναρτήσεων ενεργοποίησης στα επίπεδά τους. Αυτό τους επιτρέπει να μοντελοποιούν πολύπλοκες σχέσεις μεταξύ χαρακτηριστικών και ετικετών. Αντίθετα, το KNN και ο Κοντινότερος Κέντρου είναι γραμμικοί ταξινομητές και ενδέχεται να δυσκολεύονται να αντιληφθούν μη γραμμικά πρότυπα στα δεδομένα.
- Τα νευρωνικά δίκτυα συχνά ωφελούνται από μεγάλα σύνολα δεδομένων. Έχουν τη δυνατότητα να μαθαίνουν από ένα μεγάλο αριθμό παραδειγμάτων και να γενικεύουν καλά σε μη ορατά δεδομένα. Αντίθετα, ο KNN, που βασίζεται σε ολόκληρο το σύνολο δεδομένων κατά την πρόβλεψη, ενδέχεται να μην κλιμακώνεται καλά με ένα μεγάλο αριθμό παραδειγμάτων.
- Το KNN έχει υπερπαραμέτρους, όπως ο αριθμός των γειτόνων (k), που πρέπει να ρυθμιστούν. Η επιλογή του k μπορεί να επηρεάσει την απόδοση, και η εύρεση της βέλτιστης τιμής απαιτεί πειραματισμό. Τα νευρωνικά δίκτυα έχουν επίσης υπερπαραμέτρους, αλλά μπορεί να είναι πιο συγκρίσιμες ή πιο εύκολες να ρυθμιστούν σε ορισμένα σενάρια.
- Τα RBF Neural Networks είναι ικανά να μοντελοποιούν πιο πολύπλοκες συναρτήσεις, καθώς είναι σε θέση να μάθουν περίπλοκα χαρακτηριστικά σε υψηλές διαστάσεις. Εάν οι εικόνες του CIFAR-10 απαιτούν την αναγνώριση σύνθετων σχημάτων και συνδυασμών χαρακτηριστικών, το RBF Neural Network μπορεί να επωφεληθεί από την ικανότητά του να αναπαριστά περίπλοκα πρότυπα.